

MScFE 690 Capstone -Group 17843

Authors: Edgar NAVA , edgar.nava@growwm.com & Celestin NYANDWI ,
celestinyandwi48@gmail.com

✓ VIX Futures CBOE F1 F2

This notebook downloads monthly VX futures directly from Cboe and creates

`VIX_Futures_Term_Structure_F1_F2.csv` with:

`Date, F1, F2, Days_to_exp1, Days_to_exp2`

The study period runs from the start of the futures market on **March 26, 2004 through September 16, 2026**.

Definitions

- F1 and F2 use Cboe's daily `Settle` prices for the first two monthly contracts available and still outstanding at the end of each observation date. Weekly contracts are excluded. The initial contract listings did not cover every calendar month.
- A contract becomes available in this dataset on its first positive Cboe settlement. Leading zero-price placeholders are ignored; subsequent missing prices are not filled or replaced with another contract.
- Prices before March 26, 2007 are divided by ten to express the full series in the same volatility-point units, following [Cboe's contract rescaling](#).
- The day counts use the **VolChart.io convention**: third Friday of the month after the contract month, minus 31 calendar days, minus the observation date. This is applied to the actual contracts selected; it is not the official Cboe expiration date or a verified last trading date.

Sources: [Cboe historical futures data](#) and [Cboe settlement archive](#).

✓ 1. Install the library

```
%pip -q install "pandas>=2.0,<3.0"
```

✓ 2. Load the download functions

"""Download monthly VIX futures from Cboe for the fixed Capstone period.

Output: VIX_Futures_Term_Structure_F1_F2.csv

Columns: Date, F1, F2, Days_to_exp1, Days_to_exp2

F1/F2 are the first two available monthly VX contracts still outstanding at the end of each date. Prices are Cboe's daily Settle values, in volatility percentage. Weekly contracts and the morning final settlement of an expired contract are excluded.

Day counts follow the VolChart formula:

third Friday of the following month minus 31 calendar days minus Date.

Sources:

<https://www.cboe.com/markets/us/futures/market-statistics/historical-data/futures>

<https://www.cboe.com/markets/us/futures/market-statistics/historical-data/settlement>

<https://volchart.io/> (day-count convention only)

Requirements: Python 3.10+, pandas>=2.0.

"""

```
from __future__ import annotations
```

```
import csv
```

```
import io
```

```
import json
```

```
import math
```

```
import time
```

```
from datetime import date, timedelta
```

```
from decimal import Decimal
```

```
from pathlib import Path
```

```
from urllib.error import HTTPError, URLError
```

```
from urllib.request import Request, urlopen
```

```
import pandas as pd
```

```
# The study period remains fixed throughout the Capstone.
```

```
START_DATE = "2004-03-26"
```

```
END_DATE = "2026-09-16"
```

```
OUTPUT_FILENAME = "VIX_Futures_Term_Structure_F1_F2.csv"
```

```
MANIFEST_URL = (
```

```
    "https://www-api.cboe.com/us/futures/market_statistics/"
```

```
    "historical_data/product/list/VX/"
```

```
)
```

```
CALENDAR_URL = (
```

```
    "https://cdn.cboe.com/data/us/futures/market_statistics/"
```

```

    "historical_data/cfevoloi.csv"
)
ARCHIVE_BASE = "https://cdn.cboe.com/resources/futures/archive/volume-and-price"
MONTH_CODES = "FGHJKMNQUVXZ"
OUTPUT_COLUMNS = ["Date", "F1", "F2", "Days_to_exp1", "Days_to_exp2"]
RESCALING_DATE = pd.Timestamp("2007-03-26")
# Listed contract months in Cboe's settlement archive; monthly listings were
# not continuous in the first two years. Other missing files remain errors.
EARLY_CONTRACT_MONTHS = {
    2004: {5, 6, 7, 8, 9, 10, 11},
    2005: {1, 2, 3, 5, 6, 8, 10, 11, 12},
}

def download_file(url: str, destination: Path) -> bytes:
    """Download once, with bounded retries; retain raw files for reproducibilit
    if destination.exists():
        return destination.read_bytes()
    destination.parent.mkdir(parents=True, exist_ok=True)
    for attempt in range(3):
        try:
            request = Request(url, headers={"User-Agent": "WQU-Capstone-Researc
            with urlopen(request, timeout=60) as response:
                content = response.read()
            temporary = destination.with_suffix(destination.suffix + ".tmp")
            temporary.write_bytes(content)
            temporary.replace(destination)
            return content
        except HTTPError as error:
            # Do not repeatedly request forbidden or nonexistent files.
            if error.code not in (429, 500, 502, 503, 504) or attempt == 2:
                raise RuntimeError(f"Cboe download failed: {url} ({error})") fr
            except (URLError, TimeoutError) as error:
                if attempt == 2:
                    raise RuntimeError(f"Cboe download failed: {url} ({error})") fr
                time.sleep(2 ** attempt)
            raise RuntimeError(f"No data downloaded from {url}")

def read_cboe_csv(content: bytes, first_column: str) -> pd.DataFrame:
    """Read Cboe headers and ignore only surplus empty trailing CSV fields."""
    rows = list(csv.reader(io.StringIO(content.decode("utf-8-sig"))))
    for number, row in enumerate(rows):
        if row and row[0].strip() == first_column:
            header = [column.strip() for column in row]
            width = len(header)
            cleaned = []
            for values in rows[number + 1:]:
                if not values or not any(value.strip() for value in values):
                    continue
                # Some 2004/2005 archive rows have an extra trailing comma.

```

```

        if len(values) > width and not any(value.strip() for value in v
            values = values[:width]
        if len(values) > width:
            raise ValueError("Unexpected field count in Cboe CSV.")
        values += [""] * (width - len(values))
        cleaned.append(values)
    return pd.DataFrame(cleaned, columns=header)
raise ValueError(f"Cboe response has no {first_column!r} CSV header.")

def add_months(year: int, month: int, count: int) -> tuple[int, int]:
    total = year * 12 + month - 1 + count
    new_year, index = divmod(total, 12)
    return new_year, index + 1

def volchart_convention_date(year: int, month: int) -> date:
    """VolChart's display date for this contract month; not an exchange calendar
    next_year, next_month = add_months(year, month, 1)
    first_day = date(next_year, next_month, 1)
    third_friday = first_day + timedelta(days=(4 - first_day.weekday()) % 7 + 1)
    return third_friday - timedelta(days=31)

def load_contract(content: bytes, contract_month: str) -> pd.DataFrame:
    frame = read_cboe_csv(content, "Trade Date")
    required = {"Trade Date", "Futures", "Settle"}
    if not required.issubset(frame.columns):
        raise ValueError(f"Missing fields in {contract_month}: {required - set(
            frame["Date"] = pd.to_datetime(frame["Trade Date"], format="mixed", errors=
            if frame["Date"].isna().any() or frame["Date"].duplicated().any():
                raise ValueError(f"Missing or duplicate dates in {contract_month}")
            frame["Settle"] = pd.to_numeric(frame["Settle"], errors="raise")
            if not frame["Settle"].dropna().map(math.isfinite).all():
                raise ValueError(f"Nonfinite settlement prices in {contract_month}")
            # Zero placeholders are not tradable futures prices. Never replace with Clc
            frame.loc[frame["Settle"] <= 0, "Settle"] = float("nan")
            # CFE IC07-03: earlier quotes used ten times today's volatility-point scale
            # Apply by observation date, including contracts that expired after the cha
            earlier = frame["Date"].lt(RESCALING_DATE) & frame["Settle"].notna()
            frame.loc[earlier, "Settle"] = frame.loc[earlier, "Settle"].map(
                lambda value: float(Decimal(str(value)) / Decimal("10"))
            )
    return frame.sort_values("Date").reset_index(drop=True)

def retrieve_history(output_dir: str | Path = ".", cache_dir: str | Path | None
    """Download monthly Cboe futures, save the five-column CSV, and return it."""
    output_dir = Path(output_dir).expanduser()
    output_dir.mkdir(parents=True, exist_ok=True)
    cache = Path(cache_dir) if cache_dir else output_dir / "_cboe_cache"

```

```

first, last = pd.Timestamp(START_DATE), pd.Timestamp(END_DATE)

calendar_bytes = download_file(CALENDAR_URL, cache / "cfevoloi.csv")
calendar_frame = read_cboe_csv(calendar_bytes, "Date")
dates = pd.to_datetime(calendar_frame["Date"], format="mixed", errors="raise")
dates = dates.loc[dates.between(first, last)].sort_values().reset_index(drop=True)
if dates.empty or dates.duplicated().any() or dates.min() != first or dates.max() != last:
    raise ValueError("Cboe calendar is incomplete for the fixed study period")

manifest_bytes = download_file(MANIFEST_URL, cache / "VX_manifest.json")
manifest = json.loads(manifest_bytes)
modern = {}
for records in manifest.values():
    for record in records:
        if record.get("duration_type") != "M" or record.get("futures_root") != "":
            continue
        month = record["expire_date"][:7]
        if month in modern:
            raise ValueError(f"Ambiguous monthly VX contract in Cboe manifest")
        modern[month] = record

contracts = []
# Two additional months cover F2 after a roll in the last study month.
final_year, final_month = add_months(last.year, last.month, 2)
for period in pd.period_range(first.strftime("%Y-%m"), f"{final_year}-{final_month}"):
    month = str(period)
    if period.year in EARLY_CONTRACT_MONTHS and period.month not in EARLY_CONTRACT_MONTHS:
        continue
    if period.year <= 2013:
        # Archive files retain early-2013 prices missing from the modern files
        name = f"CFE_{MONTH_CODES[period.month - 1]}{period.year % 100:02d}"
        url = ARCHIVE_BASE + name
        content = download_file(url, cache / name)
        frame = load_contract(content, month)
        # Expired archive files include a final-settlement row. Its date is
        # the official maturity boundary, not necessarily the last trade date
        expiration = frame["Date"].max()
        if expiration.to_period("M") != period:
            raise ValueError(f"Unexpected archive maturity for {month}: {expiration}")
        if month in modern and expiration != pd.Timestamp(modern[month]["expiration"]):
            raise ValueError(f"Archive/manifest maturity disagreement for {month}")
    else:
        if month not in modern:
            raise ValueError(f"Monthly VX contract not listed by Cboe: {month}")
        record = modern[month]
        url = "https://cdn.cboe.com/" + record["path"].lstrip("/")
        name = Path(record["path"]).name
        content = download_file(url, cache / name)
        frame = load_contract(content, month)
        expiration = pd.Timestamp(record["expire_date"])
# Leading zero-settlement rows are archive placeholders. Availability s

```

```

# at the first positive settlement; later gaps are never skipped.
first_date = frame.loc[frame["Settle"].notna(), "Date"].min()
if pd.isna(first_date):
    raise ValueError(f"No positive Cboe settlement for {month}")
contracts.append({"month": month, "expiration": expiration,
                  "first_date": first_date, "frame": frame})

schedule = pd.DataFrame([
    {"month": c["month"], "expiration": c["expiration"], "first_date": c["f
    for c in contracts
])
schedule = schedule.sort_values("expiration").reset_index(drop=True)
if schedule["month"].duplicated().any() or schedule["expiration"].duplicate
    raise ValueError("Duplicate monthly contracts or maturities.")
indexed = {c["month"]: c["frame"].set_index("Date")["Settle"] for c in cont

output_rows = []
for observation_date in dates:
    # Rank from the contract schedule, not from whichever price happens to
    # be available. A missing F1 must never promote F2 into the F1 column.
    nearby = schedule.loc[
        schedule["expiration"].gt(observation_date)
        & schedule["first_date"].le(observation_date)
    ].head(2)
    if len(nearby) != 2:
        raise ValueError(f"Two monthly contracts are unavailable on {observ
    row = {"Date": observation_date}
    for number, contract in enumerate(nearby.itertuples(index=False), 1):
        month = contract.month
        year_number, month_number = map(int, month.split("-"))
        convention_date = volchart_convention_date(year_number, month_numbe
        row[f"F{number}"] = indexed[month].get(observation_date, float("nar
        row[f"Days_to_exp{number}"] = (convention_date - observation_date.c
    output_rows.append(row)

data = pd.DataFrame(output_rows, columns=OUTPUT_COLUMNS)
if data[["F1", "F2"]].isna().any().any():
    missing = data.loc[data[["F1", "F2"]].isna().any(axis=1), "Date"]
    raise ValueError(f"Cboe has missing nearby prices on {missing.dt.strfti
if not (data["Days_to_exp1"].ge(0) & data["Days_to_exp2"].gt(data["Days_to_
    raise ValueError("Invalid day counts or contract ordering.")
# Save settlement prices in a consistent volatility-point scale.
data.to_csv(output_dir / OUTPUT_FILENAME, index=False, date_format="%Y-%m-%
print(f"Saved {len(data):,} dates, {START_DATE} to {END_DATE}")
return data

```

✓ 3. Download and save the data

The CSV is saved in the current working directory. The `_cboe_cache` folder keeps the raw Cboe downloads for reuse.

```
OUTPUT_DIR = Path.cwd()
data = retrieve_history(OUTPUT_DIR)
data.head()
```

Saved 5,656 dates, 2004-03-26 to 2026-09-16

	Date	F1	F2	Days_to_exp1	Days_to_exp2
0	2004-03-26	20.32	20.16	53	81
1	2004-03-29	19.80	19.77	50	78
2	2004-03-30	19.62	19.79	49	77
3	2004-03-31	19.75	19.85	48	76
4	2004-04-01	19.62	19.61	47	75